

RELAZIONE PROJECT WORK – JUPITER NETWORK SIMULATION

1. Introduzione

Questa relazione è nata con l'intento di simulare il comportamento di un data center e, in particolare, ho cercato di basarmi sulla topologia di Jupiter che è un'architettura di rete usata nei Data Center di Google. A differenza di quest'ultima, che lavora tramite protocolli di routing a livello 3 dello stack OSI, ho provato a lavorare il più possibile a livello 2 con gli indirizzi MAC.

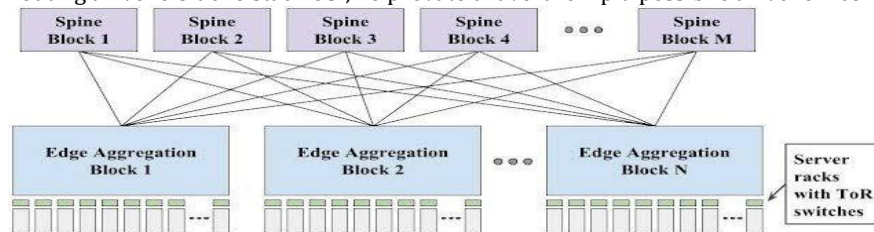


Figura 1 Jupiter

Osservando bene l'architettura si nota subito una problematica non da poco: i possibili loop (in seguito verrà riportata la soluzione) soprattutto nel caso di messaggi broadcast come ARP request che sarebbero ovviamente utilizzati, dagli switch, per l'apprendimento e il popolamento delle proprie tabelle di instradamento. Proprio a tal proposito si è scelto ovviamente di utilizzare la flessibilità che SDN mette a disposizione nella modifica delle funzionalità e del comportamento tramite modifiche software di un dispositivo di instradamento che è possibile definire "generici switch SDN" e a cui mi riferirò durante la trattazione come "elementi di rete, switch, datapath" o, talvolta, specificandone il comportamento.

Ho poi provato ad implementare quante più funzionalità possibili per rendere il funzionamento il più vicino a quello reale implementando, quindi, un controllo degli accessi, in modo tale da ottenere una parte degli pc server dedicati al cloud computing privato e altri a quello pubblico.

Proseguendo poi il lavoro con la risposta ad attacchi DoS e load balancing nella scelta dei collegamenti spine.

2. Obiettivi

Gli obiettivi già citati sono stati:

- Simulare traffico in un'architettura di un datacenter
- Implementare un controllo degli accessi per differenziare cloud pubblico e privato
- Gestire attacchi DoS simulando traffico UDP e TCP malevolo che provasse a congestionare un link
- Gestire il fenomeno del Broadcast Storm

3. Attività svolte

Innanzitutto, ho progettato la topologia considerando vari aspetti per la simulazione, quali: la banda disponibile su ogni collegamento, un numero di collegamenti e di elementi di smistamento abbastanza esiguo e un numero di server elevato per poter differenziare quelli ad accesso libero e non. Ho poi scelto un elemento di rete detto Border Leaf, da utilizzare anche come gateway per permettere l'accesso agli utenti esterno.

```
class datacenter(object):
    def __init__(self):
        "Create a data center network"
        self.net = Mininet(controller=RemoteController, link=TCLink) #utilizzerò quindi un controller esterno come ryu e userò
        traffic control di linux per simulare la banda sui collegamenti virtuali
        info("-avvio del controller \n")
        c1 = self.net.addController('c1', controller=RemoteController) #senza indirizzo ip è sottinteso localhost
        c1.start()
        info("-creazione host \n")
        self.h1 = self.net.addHost('h1', mac='00:00:00:00:00:01', ip='10.0.0.1')
        #####
        self.h8 = self.net.addHost('h8', mac='00:00:00:00:00:08', ip='10.0.0.8')
        self.utente1 = self.net.addHost('utente1', mac='00:00:00:00:00:09', ip='10.0.0.9')
        #####
        self.utenteaut2 = self.net.addHost('utenteaut2', mac='00:00:00:00:00:13', ip='10.0.0.13')
        info("-creo gli switch spine \n")
        self.spine1 = self.net.addSwitch('s1', cls=OVSKernelSwitch)
        #####
        info("-creo gli switch leaf \n")
        self.leaf1 = self.net.addSwitch('s5', cls=OVSKernelSwitch)
        #####
        info("-collegamento host con switch leaf(per prova qui utilizzo 10Mbps) \n")
        self.net.addLink(self.h1, self.leaf1, bw=10)
        self.net.addLink(self.h2, self.leaf1, bw=10)
        self.net.addLink(self.h3, self.leaf2, bw=10)
        #####
        self.net.addLink(self.utente1, self.leaf1, bw=10)
        #####
        self.net.addLink(self.utenteaut2, self.leaf1, bw=10)
        info("-collegamento leaf1 e spine full mesh(qui uso collegamenti a 100Mbps) \n")
```

```

self.net.addLink(self.leaf1, self.spine1, bw=100)
self.net.addLink(self.leaf1, self.spine2, bw=100)
self.net.addLink(self.leaf1, self.spine3, bw=100)
self.net.addLink(self.leaf1, self.spine4, bw=100)
info("-collegamento leaf2 e spine in full mesh(qui uso collegamenti a 100Mbps) \n")
/////
info("-collegamento leaf3 e spine in full mesh(qui uso collegamenti a 100Mbps) \n")
/////
info("-collegamento leaf4 e spine in full mesh(qui uso collegamenti a 100Mbps) \n")
/////
info("-avvio rete\n")
self.net.build() #assemblo prima e poi avvio
self.net.start()

if __name__ == '__main__':
    setLogLevel('info')
    env = datacenter()
    CLI(env.net) #avvio la cli di mininet per gestire la rete

```

Viene qui riportato il codice (eliminando linee di codice di pattern facilmente individuabili e ripetitivi e sostituiti da ///). Ho iniziato quindi con la definizione della classe ridefinendone il costruttore; qui ho specificato l'utilizzo di un controller esterno (è stato poi utilizzato Ryu) e che venisse utilizzato il Traffic Control di linux per limitare la banda dei link e simularne il comportamento.

Dopo aver aggiunto e avviato il controller ho iniziato a creare gli host (PC); quest'ultimi si dividono in 2 tipi: server ed utenti esterni che vogliono accedere al data center.

Dopo aver creato la topologia ho ovviamente verificato che i link fossero funzionanti senza problemi dalla CLI di mininet usando i comandi net, dumps e links(di fianco è riportato l'output del comando links).

La rete si presenta formata da:

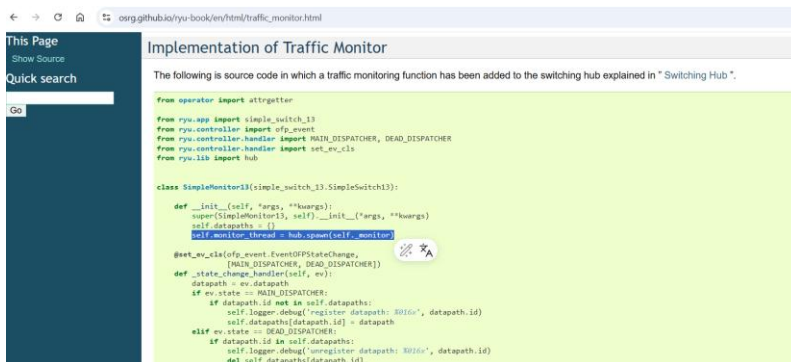
- switch leaf vicini ai pc server (2 per ogni leaf) tra cui il border leaf che fungeva anche da gateway per l'accesso dall'esterno.
- Ogni leaf collegato a tutti gli elementi di tipo Spine attraverso dei collegamenti con banda maggiore
- Degli host che simulano PC che sono o esterni o dei server con cui comunicare

Successivamente si è passati alla creazione del controller che si è rivelata articolata a causa delle varie funzionalità, ma la documentazione utilizzata si è rivelata di supporto per l'implementazione di quest'ultime, in particolare per il traffic monitoring.

Sono partito dall'implementazione iniziale del controller switch13 e ho modificato lo script python per modificare il comportamento quel semplice controller e renderlo capace di estendere le funzionalità di tutti gli switch generici per poter lavorare anche al livello 3 dello stack OSI.

Ho innanzitutto creato ulteriori dizionari per:

- gestire i diversi switch e tenerne traccia (come a crearne delle memorie per ognuno)
- tener traccia del traffico di bytes per calcolare il throughput
- tenere traccia dei tempi per la stessa motivazione di sopra.



messaggi troncati e non inviati completamente tipico delle versioni di OVS precedenti alla v2.1.0. Il tutto è stato effettuato con operazioni che saranno ricorrenti per recuperare parser, datapath e ofproto (per le costanti e strutture dei messaggi) al fine di applicare al traffico che matcha il filtro applicato (in questo caso tutto) delle actions ed installare la regola attraverso la funzione add_flow creata in seguito. A quest'ultima, a differenza di quella riportata nelle slide, è stato aggiunto un hard_timeout affinché il datapath potesse eliminare delle regole a scelta (utile nel caso volessimo eliminare dopo tot secondi le restrizioni per la gestione degli attacchi DoS creando, quindi, un meccanismo di autorecovery dall'isolamento dell'utente malevolo). Ho poi implementato il monitor che richiede le statistiche attraverso un metodo che manda, ogni 3 secondi, un messaggio Open Flow di tipo StatsRequest per ogni elemento del dizionario dei datapath (gli switch instradatori di traffico).

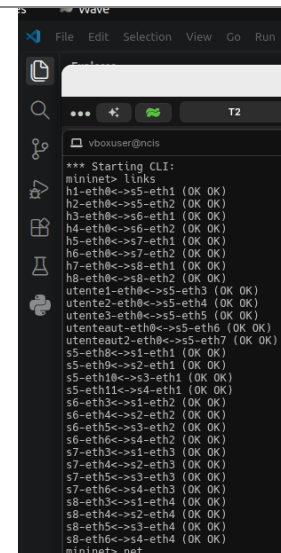


Figura 2 output per links

Il lavoro è proseguito poi, basandomi sulla documentazione, con il traffic monitoring e la creazione del monitor con dei thread tramite la libreria thread messa a disposizione da ryu (che crea dei thread in modo molto più efficiente e veloce).

Ho poi iniziato con la gestione degli eventi StateChange per eliminare le connessioni DEAD, alleggerire il lavoro del mio monitor e liberare memoria, per poi passare poi alla gestione degli eventi SwitchFeature nello stato CONFIG.

Questi si verificano, quindi, all'accensione in fase di configurazione ed ho qui risolto (come riportato nelle slide del corso) il bug noto dei

L'implementazione del controller è poi proseguita con la gestione dell'evento PortStatsReply verificatosi nella modalità MAIN che estrae le liste di statistiche (tranne quelle della porta local riservata all'accesso allo stack di networking locale) e le carica nel dizionario insieme al tempo di acquisizione e la somma dei byte in rx e tx. Nel caso in cui non siano le prime statistiche dello switch

allora fa la differenza con le precedenti dividendo per la differenza del tempo rilevato e calcola il throughput relativo a quel collegamento. Viene riportato a video dalla CLI solamente il traffico rilevante maggiore di 0.5Mbps ed effettuato il drop dei pacchetti nel caso superi gli 8Mbps per evitare problemi di prestazioni e gestire attacchi del tipo DoS; questa soglia potrebbe essere variata, anche sperimentalmente per trovare il giusto compromesso.

Nella funzione per la gestione dei drop è stato impostato un hard_timeout per cui la regola installata in cui actions=[] (metodo per eliminare i pacchetti riportato nella documentazione) che ha il match con il filtro verrà eliminata per rendere di nuovo possibile la comunicazione a quell'utente (scelta di design della nostra implementazione del data center).

Il match per il drop dei pacchetti viene effettuato differenziando il traffico di tipo udp e di tipo tcp bloccando quindi quella tipologia di traffico che si pensa esser malevola grazie a due match differenziati.

Sono poi entrato nel cuore del controller gestendo EventsOFPPacketIn nella modalità principale di funzionamento dove, dopo aver estratto i dati e averli trasformati in packet in modo da accedere meglio alle informazioni, ho estratto l'header ethernet per recuperare i MAC address sorgente e destinazione (come riportato nelle slide) per inserirli nel corrispettivo dizionario del corrispettivo switch. Da qui in poi il lavoro si è differenziato dal classico approccio limitato al livello e dello stack e ho sfruttato

MAC_OF_VLAN_PUP	0	NO	ETHERTYPE=0x0800 or ETHERTYPE=0x86dd	ETHERTYPE=0x0800 or ETHERTYPE=0x86dd
OXM_OF_IP_DSCP	6	No	Diff Serv Code Point (DSCP). Part of the IPv4 ToS field or the IPv6 Traffic Class field.	Diff Serv Code Point (DSCP). Part of the IPv4 ToS field or the IPv6 Traffic Class field.
OXM_OF_IP_ECN	2	No	ECN bits of the IP header. Part of the IPv4 ToS field or the IPv6 Traffic Class field.	ECN bits of the IP header. Part of the IPv4 ToS field or the IPv6 Traffic Class field.
OXM_OF_IP_PROTO	8	No	IPv4 or IPv6 protocol number.	IPv4 or IPv6 protocol number.
OXM_OF_IPV4_SRC	32	Yes	IPv4 source address. Can use subnet mask or arbitrary bitmask	IPv4 source address. Can use subnet mask or arbitrary bitmask
OXM_OF_IPV4_DST	32	Yes	IPv4 destination address. Can use subnet mask or arbitrary bitmask	IPv4 destination address. Can use subnet mask or arbitrary bitmask

una volta ottenuti gli indirizzi IP, controllato il caso di accesso non autorizzato effettuando il drop dei pacchetti, ma questa volta senza eliminare la regola installata.

Altra funzionalità implementata è stata quella di creare un firewall che bloccasse il traffico udp sul porto 5001 il tutto ispezionando i pacchetti e non solo eliminando il traffico, ma installando una regola che blocchi immediatamente quest'ultimo senza raggiungere soglie.

Per gestire l'apprendimento degli indirizzi e utilizzare una strategia per evitare il broadcast storm che si sarebbe verificato nelle ARP request, ho considerato STP troppo dannoso per il throughput complessivo e deleterio per la topologia utilizzata. Ho scelto, quindi, di implementare una strategia simile allo split horizon visto nel caso di iBGP con l'annuncio di rotte apprese dai peer non client. Ho scelto quindi di effettuare flooding dei messaggi, nel caso di messaggio proveniente dallo spine verso il leaf, soltanto verso gli host. Oltre a questo controllo è stato effettuato anche un load balancing molto semplice con la scelta random nel caso in cui un leaf dovesse instradare verso uno spine per usare tutti i collegamenti ugualmente e sfruttare le risorse della rete visto che si creerebbe in questo caso uno scenario del tipo ECMP (Equal cost multi path routing).

Per rendere implementabili queste due funzionalità mi son servito di dizionari in cui, per ogni leaf, ho tenuto traccia: degli host collegati in uno e degli spine in un altro.

Riporto poi altri due accorgimenti:

- memorizzare la regola solo nel caso NON si trattasse di un messaggio flooding
- inserire anche i dati nel messaggio in caso di OFP_NO_BUFFER in cui lo switch non ha tenuto il messaggio in memoria prima di mandarlo al controller.

4. Test di funzionamento

Il lavoro è poi proseguito con i test di funzionamento, ho quindi iniziato con un ping semplice per verificare che un server potesse contattare un altro e poi ho provato lo stesso test con:

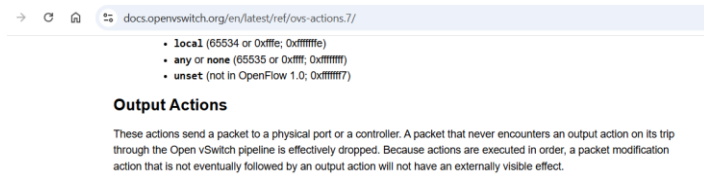
- un utente e un server pubblico
- un utente normale con un server privato
- un utente autorizzato con un server privato

Il tutto ovviamente dopo aver avviato controller ed eseguito lo script della topologia con i comandi ryu-manager datacenter_polisi.py e sudo python3 topologia.py.

Ho poi effettuato una generazione di traffico UDP da parte di un utente simulando un attacco malevolo su un link per vedere se si fosse attivato il meccanismo di controllo e gestione e per garantirmi che bloccasse il traffico in entrata da quell'utente per 120 secondi prima commentando il match del traffico udp e tcp e quindi vedendo se venisse bloccata l'intera porta rendendo indisponibile il raggiungimento di qualsiasi tipo di traffico per il dispositivo collegato a quella porta.

Ho poi effettuato il test creando traffico malevolo con iperf udp per vedere come si comportasse la mia rete, il tutto però mentre c'era uno storm di pacchetti icmp in background dallo stesso mittente per verificare se quel tipo di traffico venisse bloccato e che quindi si differenziasse il traffico malevolo UDP da quello ICMP.

Successivamente ho anche provato a congestionare il link con una generazione di traffico di tipo tcp con il comando iperf sempre e vedere il comportamento cercando di effettuare delle catture con Wireshark per notare le differenze tra il TCP Flood e quello UDP e vedere se effettivamente i pacchetti icmp venissero consegnati nonostante lo stesso utente fosse stato bloccato nel suo trasferimento di dati con TCP o UDP.



5. Risultati e Discussione

```
mininet> h1 ping -c 3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=28.3 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=1.31 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.084 ms

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 0.084/9.914/28.345/13.042 ms
```

```
vboxuser@ncis:~/Documents$ ryu-manager datacenter_polisi.py
loading app datacenter_polisi.py
loading app ryu.controller.ofp_handler
instantiating app datacenter_polisi.py of datacenter_polisi
instantiating app ryu.controller.ofp_handler of OFPHandler
ACCESSO NON AUTORIZZATO di 10.0.0.9 verso il server privato 10.0.0.8
```

```
mininet> h1 ping -c 3 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=29.8 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=2.92 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.203 ms

--- 10.0.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2104ms
rtt min/avg/max/mdev = 0.203/10.977/29.813/13.365 ms
```

```
mininet> utente1 ping -c 3 h8
PING 10.0.0.8 (10.0.0.8) 56(84) bytes of data.
64 bytes from 10.0.0.8: icmp_seq=1 ttl=64 time=34.7 ms
64 bytes from 10.0.0.8: icmp_seq=2 ttl=64 time=1.02 ms
64 bytes from 10.0.0.8: icmp_seq=3 ttl=64 time=1.66 ms

--- 10.0.0.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2047ms
rtt min/avg/max/mdev = 1.021/12.463/34.712/15.734 ms
```

```
mininet> h4 iperf -s -u 6
mininet> utente3 iperf -c h4 -u -t 20 -b 100M

-----
Client connecting to 10.0.0.4, UDP port 5001
Sending 1470 byte datagrams, IPG target: 112.15 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.11 port 42532 connected with 10.0.0.4 port 5001
[ 10] Interval Transfer Bandwidth
[ 1] 0.0000-20.0001 sec 113 MBytes 47.3 Mbits/sec
[ 1] Sent 80510 datagrams
[ 3] WARNING: did not receive ack of last datagram after 10 tries.
mininet> utente3 ping -c 3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2032ms
```

```
Switch 6, Porta 5: Throughput = 3.36 Mbps
Switch 6, Porta 2: Throughput = 3.36 Mbps
Switch 3, Porta 1: Throughput = 3.36 Mbps
Switch 3, Porta 2: Throughput = 3.36 Mbps
Switch 5, Porta 10: Throughput = 3.37 Mbps
Switch 5, Porta 5: Throughput = 3.37 Mbps
Switch 6, Porta 5: Throughput = 8.17 Mbps
!!! ANOMALIA RILEVATA !!! Switch 6 Porta 5 supera gli 8 Mbps. Esegui DROP.
Switch 6, Porta 2: Throughput = 8.17 Mbps
!!! ANOMALIA RILEVATA !!! Switch 6 Porta 2 supera gli 8 Mbps. Esegui DROP.
Switch 3, Porta 1: Throughput = 8.17 Mbps
!!! ANOMALIA RILEVATA !!! Switch 3 Porta 1 supera gli 8 Mbps. Esegui DROP.
Switch 3, Porta 2: Throughput = 8.17 Mbps
!!! ANOMALIA RILEVATA !!! Switch 3 Porta 2 supera gli 8 Mbps. Esegui DROP.
Switch 5, Porta 10: Throughput = 8.17 Mbps
!!! ANOMALIA RILEVATA !!! Switch 5 Porta 10 supera gli 8 Mbps. Esegui DROP.
Switch 5, Porta 5: Throughput = 8.17 Mbps
!!! ANOMALIA RILEVATA !!! Switch 5 Porta 5 supera gli 8 Mbps. Esegui DROP.
Switch 5, Porta 5: Throughput = 6.89 Mbps
Switch 5, Porta 5: Throughput = 3.98 Mbps
```

```
h8 h8-eth0:s8-eth2
utente1 utente1-eth0:s5-eth3
utente2 utente2-eth0:s5-eth4
```

```
Switch 3, Porta 2: Throughput = 9.51 Mbps
!!! ANOMALIA RILEVATA !!! Switch 3 Porta 2 supera gli 8 Mbps. Esegui DROP.
Switch 6, Porta 1: Throughput = 9.35 Mbps
!!! ANOMALIA RILEVATA !!! Switch 6 Porta 1 supera gli 8 Mbps. Esegui DROP.
Switch 6, Porta 5: Throughput = 9.34 Mbps
!!! ANOMALIA RILEVATA !!! Switch 6 Porta 5 supera gli 8 Mbps. Esegui DROP.
Switch 3, Porta 1: Throughput = 9.32 Mbps
!!! ANOMALIA RILEVATA !!! Switch 3 Porta 1 supera gli 8 Mbps. Esegui DROP.
Switch 3, Porta 2: Throughput = 9.32 Mbps
!!! ANOMALIA RILEVATA !!! Switch 3 Porta 2 supera gli 8 Mbps. Esegui DROP.
Switch 5, Porta 10: Throughput = 9.32 Mbps
!!! ANOMALIA RILEVATA !!! Switch 5 Porta 10 supera gli 8 Mbps. Esegui DROP.
Switch 5, Porta 3: Throughput = 9.33 Mbps
!!! ANOMALIA RILEVATA !!! Switch 5 Porta 3 supera gli 8 Mbps. Esegui DROP.
```

```
mininet> utente1 ping -c 3 h8
PING 10.0.0.8 (10.0.0.8) 56(84) bytes of data.

--- 10.0.0.8 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2673ms
```

Il primi due test del ping verso un server pubblico è andato a buon fine come ci si aspettava, con un 0% di packet loss.

Il terzo, come da previsioni ha avuto un 100% di packet loss visto che è stato effettuato un tentativo non valido di ping da un utente non autorizzato ad un server privato ed è stato possibile notarlo anche

nell'output del controller che ha risposto correttamente mostrando l'output atteso.

Il test n4 alla mostra come in realtà l'utente autorizzato possa raggiungere il server privato normalmente ed anche questo ha avuto esito positivo.

Passando poi alla gestione degli attacchi DoS ho provato il test 5 aspettandomi che venissero eliminati tanti pacchetti e che l'utente non autorizzato fosse stato classificato come malevolo e quindi venisse applicata la regola action=[] di drop dei pacchetti alla porta associata al suo collegamento con la rete ed effettivamente è stato così.

Il tutto provato da un tentativo di raggiungere un altro server con un ping successivo che è fallito e anche dall'output del controller che ha tenuto traccia della saturazione della banda su quel link.

Per utilizzare il tool Wireshark ho innanzitutto individuato tramite il comando net la porta di cui dovessi catturare il traffico e ho configurato il tool affinché lo facesse.

Successivamente ho iniziato la tempesta infinita di ping da utente1 a h3 e successivamente generato un traffico UDP a un throughput maggiore della banda del collegamento per vedere come si sarebbe comportato.

Vedendo il risultato di Wireshark e la console del controller è possibile notare come il traffico sia stato controllato eliminando i pacchetti e come i pacchetti ICMP continuassero ad essere mandati nonostante fosse dopo che la regola di eliminazione di quei pacchetti fosse stata installata nello switch.

Questo mostra come gli altri tipi di traffico su quella porta ormai siano indipendenti e che il blocco riguardi anche altro oltre che la porta di ingresso dello switch utilizzato.

I pacchetti ICMP sono visibili nel report con un colore viola.

Nel caso di traffico TCP che provi a saturare il link il risultato si è mostrato il medesimo e lo confermano anche i messaggi di errore per la saturazione del collegamento che vengono mostrati a video dal controller.

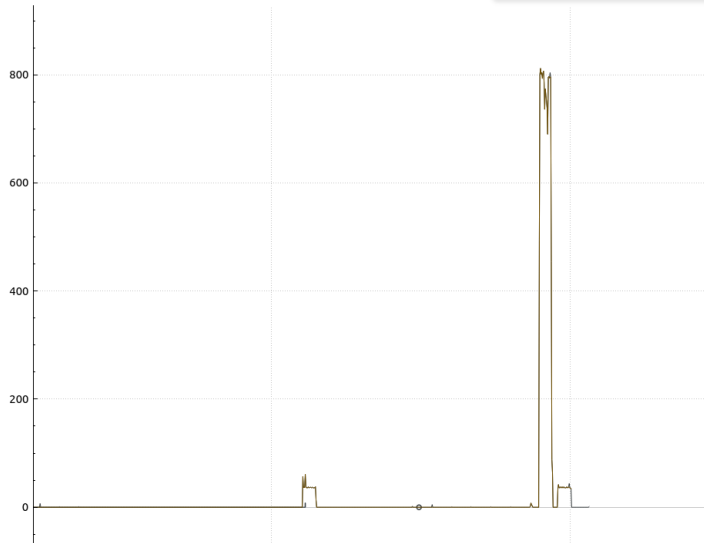
La rete quindi ha mostrato grande flessibilità, anche se utilizzando dei messaggi di altro tipo si sarebbe potuto effettuare un controllo a livello di flusso, ma di ciò ne parlerò nella prossima sezione.



```
mininet> h3 iperf -s -u -p 8888 &
mininet> utente1 ping h3 &
mininet> utente1 iperf -c h3 -u -b 50M -t 20 -p 8888
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=29.3 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.517 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.107 ms
```

16430	209.239130120	10.0.0.9	10.0.0.3	UDP	1512	36542	→	8888	Len=1470
16431	209.249984260	10.0.0.9	10.0.0.3	UDP	1512	36542	→	8888	Len=1470
16432	209.260890396	10.0.0.9	10.0.0.3	UDP	1512	36542	→	8888	Len=1470
16433	209.896442349	10.0.0.9	10.0.0.3	ICMP	98	Echo (ping) request	id=0xb96f, seq=84/21504, ttl=64 (reply in 16434)		
16434	209.896841410	10.0.0.3	10.0.0.9	ICMP	98	Echo (ping) reply	id=0xb96f, seq=84/21504, ttl=64 (request in 16433)		
16435	210.921413671	10.0.0.9	10.0.0.3	ICMP	98	Echo (ping) request	id=0xb96f, seq=85/21760, ttl=64 (reply in 16436)		
255	185.760890197	10.0.0.9	10.0.0.3	ICMP	98	Echo (ping) request	id=0xb96f, seq=60/15360, ttl=64 (reply in 256)		
256	185.766951050	10.0.0.9	10.0.0.3	ICMP	98	Echo (ping) reply	id=0xb96f, seq=60/15360, ttl=64 (request in 255)		
257	186.791202697	10.0.0.9	10.0.0.3	ICMP	98	Echo (ping) request	id=0xb96f, seq=61/15616, ttl=64 (reply in 258)		
258	186.791321738	10.0.0.3	10.0.0.9	ICMP	98	Echo (ping) reply	id=0xb96f, seq=61/15616, ttl=64 (request in 257)		
259	186.949798381	10.0.0.9	10.0.0.3	UDP	1512	36542	→	8888	Len=1470
260	186.950361306	10.0.0.9	10.0.0.3	UDP	1512	36542	→	8888	Len=1470
261	186.951362355	10.0.0.9	10.0.0.3	UDP	1512	36542	→	8888	Len=1470

17751 1575.2653592..	10.0.0.9	10.0.0.3	TCP	65226 36892 → 5001	[PSH, ACK] Seq=26107501 Ack=29 Win=42496 Len=65160 TSval=3510431118 TSecr=4220835540
17752 1575.2654413..	10.0.0.3	10.0.0.9	TCP	66 5001 → 36892	[ACK] Seq=29 Ack=26172661 Win=836608 Len=0 TSval=4220835649 TSecr=3510431118
17753 1575.3198781..	10.0.0.9	10.0.0.3	TCP	41866 36892 → 5001	[FIN, PSH, ACK] Seq=26172661 Ack=29 Win=42496 Len=41800 TSval=3510431173 TSecr=4220835595
17754 1575.3199592..	10.0.0.3	10.0.0.9	TCP	66 5001 → 36892	[ACK] Seq=29 Ack=26214462 Win=857088 Len=0 TSval=4220835704 TSecr=3510431173
17755 1575.3272196..	10.0.0.3	10.0.0.9	TCP	66 5001 → 36892	[FIN, ACK] Seq=29 Ack=26214462 Win=857088 Len=0 TSval=4220835711 TSecr=3510431173
17756 1575.3554283..	10.0.0.9	10.0.0.3	TCP	66 36892 → 5001	[ACK] Seq=26214462 Ack=30 Win=42496 Len=0 TSval=3510431289 TSecr=4220835711



Il grafico affianco mostra i picchi di traffico e soprattutto di pacchetti al secondo che sono stati inviati durante la generazione del traffico. Si nota facilmente come UDP, che è rappresentato dalla curva che ha il picco più alto, ha mostrato un picco, appunto, molto più grande di quello tcp a prova della differenza tra i due protocolli di trasporto. Mentre TCP ha dei controlli e limita il numero di pacchetti nel caso di congestione e ha un slow start che poi man mano aumenta l'altro non ha limitazioni e cerca di monopolizzare l'utilizzo di banda disponibile. Questo cerca di inondare il link e inviare quanti più pacchetti possibili.

6. Conclusioni

I test hanno dato esito positivo, il tutto è andato come atteso, nonostante sarebbero possibili tanti miglioramenti.

Il paradigma SDN si sia rilevato molto adatto all'amministrazione di una rete di un server del tipo spine-leaf disaccoppiando control plane e data plane e rendendo i semplici dispositivi di inoltramento molto più complessi.

Proprio questa semplicità nel modificare il comportamento degli elementi di rete da spunto per tante modifiche future per migliorare questo project work.

Migliorare il filtraggio del traffico e il blocco di attacchi associando il traffico malevolo al flusso di dati e non alla porta dello switch o all'indirizzo ip e, quindi, lavorare anche con i numeri di port UDP e TCP e la tupla completa che identifica una connessione.

Le richieste e gli eventi potrebbero essere ad esempio del tipo FlowStatsRequest a differenza di quelle che ho utilizzato io per ottenere statistiche ed effettuare calcoli sul singolo flusso e cambiare, di conseguenza la soglia massima di banda massima prima di considerare quella tupla sospetta.

Si potrebbe modificare l'attuale load balancing casuale in uno dinamico che possa sfruttare appieno le StatsRequest e la gestione delle relative Reply con un nuovo monitor magari che possa gestire questo tipo di evento solo al fine di migliorare il bilanciamento delle risorse.

Altro spunto di miglioramento potrebbe essere quello di implementare algoritmi di routing all'interno della topologia e quindi rendere inutile lo split horizon grazie a meccanismi come il TTL e il controllo ad esempio del path di rete di router attraversati con meccanismi come il cluster list o originator id.

7. Bibliografia / Sitografia

https://osrg.github.io/ryu-book/en/html/traffic_monitor.html

<https://opennetworking.org/wp-content/uploads/2014/10/openflow-spec-v1.3.0.pdf>

<https://github.com/openvswitch/openflow/tree/master/of-switch>

<https://www.micahlerner.com/2022/12/13/jupiter-rising-a-decade-of-clos-topologies-and-centralized-control-in-googles-datacenter-network.html>

Slide del corso